CodeAlpha Cybersecurity Internship

# Secure Coding Review Report

Task 3 – Secure Coding Review

| | |
|---|---|
| **Application Reviewed:** | Student Portal (Flask web application) |
| **Language:** | Python 3.12 (Flask framework) |
| **Review Type:** | Manual source code review + automated static analysis (Bandit) |
| **Prepared by:** | Aysha — CodeAlpha Cybersecurity Intern |
| **Date:** | August 2026 |

## 1. Overview

This report documents a secure coding review carried out as Task 3 of the CodeAlpha Cybersecurity internship. A small Python/Flask web application, **Student Portal**, was selected as the audit target. The application simulates common real-world features — user login, password hashing, a network diagnostics endpoint, and a profile page — each of which was deliberately implemented with realistic, commonly-seen coding mistakes so the review could demonstrate a full vulnerability identification and remediation workflow.

## 2. Scope & Methodology

The review combined two complementary techniques:

**Manual inspection:** Every route handler in `app.py` was read line-by-line, tracing how user-controlled input (form fields, query parameters) flows into database queries, shell commands, hashing functions, template rendering, and deserialization calls.

**Automated static analysis:** The open-source tool **Bandit** (a Python-specific static application security testing tool) was run against the source file to independently confirm the manually identified issues and catch anything missed by eye.

Bandit's scan of the original application flagged **8 issues**: 3 High severity, 3 Medium severity, and 2 Low severity — matching the 8 vulnerabilities identified manually below.

## 3. Findings Summary

| # | Vulnerability | Severity | CWE | Location |
|---|---|---|---|---|
| 1 | Hardcoded secret key in source code | Low | CWE-798 | app.py:19 |
| 2 | SQL Injection (string-concatenated query) | High | CWE-89 | app.py:47 |
| 3 | Reflected Cross-Site Scripting (XSS) | Medium | CWE-79 | app.py:53 |
| 4 | Weak password hashing (unsalted MD5) | High | CWE-327 | app.py:62 |
| 5 | OS Command Injection | High | CWE-78 | app.py:70 |
| 6 | Server-Side Template Injection (SSTI) | High | CWE-1336 | app.py:79 |
| 7 | Insecure deserialization (pickle) | High | CWE-502 | app.py:86 |
| 8 | Debug mode / bound to all interfaces | Medium | CWE-489, 605 | app.py:93 |

# 4. Detailed Findings & Remediation

## 4.1 Hardcoded Secret Key 🟢 Low — CWE-798

**Issue:** The Flask session-signing secret key was committed directly in source code as a plain string. Anyone with repository access (or a leaked copy of the source) could forge signed session cookies.

**Recommendation:** Load the secret from an environment variable and fail startup if it is missing, so the value never lives in version control.

## 4.2 SQL Injection 🔴 High — CWE-89

**Issue:** The login query was built by concatenating the raw username and password directly into a SQL string. An attacker submitting a username like `' OR '1'='1` could bypass authentication entirely or extract arbitrary data from the database.

**Recommendation:** Use parameterized queries (placeholders) so the database driver treats user input strictly as data, never as executable SQL.

## 4.3 Reflected Cross-Site Scripting (XSS) 🟠 Medium — CWE-79

**Issue:** The username was echoed back into the HTTP response without escaping. A username containing a script tag would execute in the victim's browser.

**Recommendation:** Escape all user-controlled output before it is written into an HTML response (e.g. Flask/Jinja's autoescaping or an explicit escape() call).

## 4.4 Weak Password Hashing 🔴 High — CWE-327

**Issue:** Passwords were hashed with unsalted MD5, an algorithm that is cryptographically broken and fast to brute-force with modern hardware and rainbow tables.

**Recommendation:** Store passwords using a purpose-built, salted password hashing function (e.g. Werkzeug's generate_password_hash, which uses PBKDF2/scrypt) and compare with a constant-time check function.

## 4.5 OS Command Injection 🔴 High — CWE-78

**Issue:** A user-supplied 'host' parameter was concatenated directly into a shell command passed to os.system(). Input like `127.0.0.1; rm -rf /` would execute attacker-chosen commands on the server.

**Recommendation:** Validate that the input is actually a well-formed IP address before use, and invoke the external command as an argument list via subprocess (no shell=True), never by building a shell string.

## 4.6 Server-Side Template Injection (SSTI) 🔴 High — CWE-1336

**Issue:** A user-controlled 'name' parameter was concatenated into a template string that was then rendered. Depending on the templating engine, this can allow full remote code execution on the server.

**Recommendation:** Never build template strings from user input. Treat the value purely as data, escape it, and insert it into a fixed, pre-defined template/response.

## 4.7 Insecure Deserialization 🔴 High — CWE-502

**Issue:** The application deserialized attacker-supplied data using Python's pickle module. Pickle can execute arbitrary code during deserialization, making this endpoint a direct remote-code-execution vector.

**Recommendation:** Never unpickle untrusted input. Use a safe, data-only format such as JSON for any client-supplied data, and rely on Flask's built-in signed session cookies (or a server-side session store)

instead of custom pickle-based session handling.

### 4.8 Debug Mode Enabled / Bound to All Interfaces ● Medium — CWE-489 / CWE-605

**Issue:** The app was started with debug=True and host='0.0.0.0'. Flask's debugger exposes an interactive Python console reachable over the network, which is a direct remote-code-execution path if this configuration reaches a real deployment.

**Recommendation:** Disable debug mode outside local development, and restrict the bind address /use a reverse proxy so the app is not directly reachable on all network interfaces.

## 5. Verification of Fixes

After remediating each issue in `fixed_app/app.py`, Bandit was re-run to independently confirm the fixes.

| Metric | Original app.py | Fixed app.py |
|---|---|---|
| High severity issues | 3 | 0 |
| Medium severity issues | 3 | 0 |
| Low severity issues | 2 | 3* |

*The 3 remaining Low findings are informational notices about the use of the subprocess module in general (Bandit flags any subprocess usage for awareness). The underlying command-injection risk is fully resolved because the host is validated as a real IP address and the command is executed as an argument list, never through a shell.

## 6. General Secure Coding Recommendations

• Always use parameterized queries / an ORM for database access — never string-concatenate user input into SQL.

• Escape all output rendered into HTML, and never build template strings dynamically from user input.

• Hash passwords with a purpose-built algorithm (bcrypt/scrypt/PBKDF2) that includes a salt, never a general-purpose hash like MD5 or SHA-1.

• Avoid shell=True and raw string commands; pass arguments as a list and validate/allow-list any input used in system calls.

• Never deserialize untrusted data with pickle; prefer JSON or another data-only format.

• Keep secrets (keys, credentials, tokens) out of source code — load them from environment variables or a secrets manager.

• Disable debug mode and restrict network bind addresses before any deployment beyond local development.

• Run an automated static analyzer (e.g. Bandit for Python) as part of every code review or CI pipeline to catch regressions early.

## 7. Conclusion

The Student Portal application initially contained 8 distinct vulnerabilities spanning injection flaws, weak cryptography, insecure deserialization, and unsafe deployment configuration. All 8 were traced to their root cause through manual code review, independently confirmed with the Bandit static analyzer, and resolved in the remediated version of the application. This exercise demonstrates a complete secure coding review workflow: source review, automated scanning, documented findings with CWE references, concrete remediation, and re-verification.